



Security Audit

Kadena (RWA)

Table of Contents

Executive Summary	4
Project Context	4
Audit Scope	7
Security Rating	9
Intended Smart Contract Functions	10
Code Quality	13
Audit Resources	13
Severity Definitions	14
Status Definitions	15
Audit Findings	16
Centralisation	51
Conclusion	52
Our Methodology	53
Disclaimers	55
About Hashlock	56

CAUTION

THIS DOCUMENT IS A SECURITY AUDIT REPORT AND MAY CONTAIN CONFIDENTIAL INFORMATION. THIS INCLUDES IDENTIFIED VULNERABILITIES AND MALICIOUS CODE WHICH COULD BE USED TO COMPROMISE THE PROJECT. THIS DOCUMENT SHOULD ONLY BE FOR INTERNAL USE UNTIL ISSUES ARE RESOLVED. ONCE VULNERABILITIES ARE REMEDIATED, THIS REPORT CAN BE MADE PUBLIC. THE CONTENT OF THIS REPORT IS OWNED BY HASHLOCK PTY LTD FOR USE OF THE CLIENT.

Executive Summary

The Kadena team partnered with Hashlock to conduct a security audit of their smart contracts. Hashlock manually and proactively reviewed the code in order to ensure the project's team and community that the deployed contracts are secure.

Project Context

Kadena is a next-generation blockchain platform designed to deliver enterprise-grade scalability, security, and decentralization. Leveraging its unique multi-chain architecture, Proof of Work consensus, and EVM compatibility, Kadena enables developers to build high-performance, secure decentralized applications.

By bridging traditional enterprise needs with advanced Web3 capabilities, Kadena supports robust use cases across decentralized finance, tokenized ecosystems, and transparent governance—offering unmatched throughput, ultra-low transaction fees, and strong security guarantees.

This project includes a real-world-asset-v1 implementation in Pact, Kadena's native smart contract language. This implementation is a Kadena-native version of the ERC-3643 token standard, tailored for compliant asset tokenization.

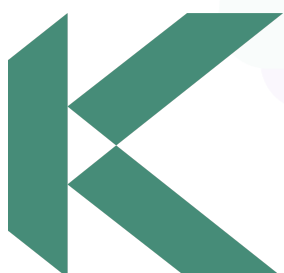
Project Name: Kadena

Project Type: RWA Token

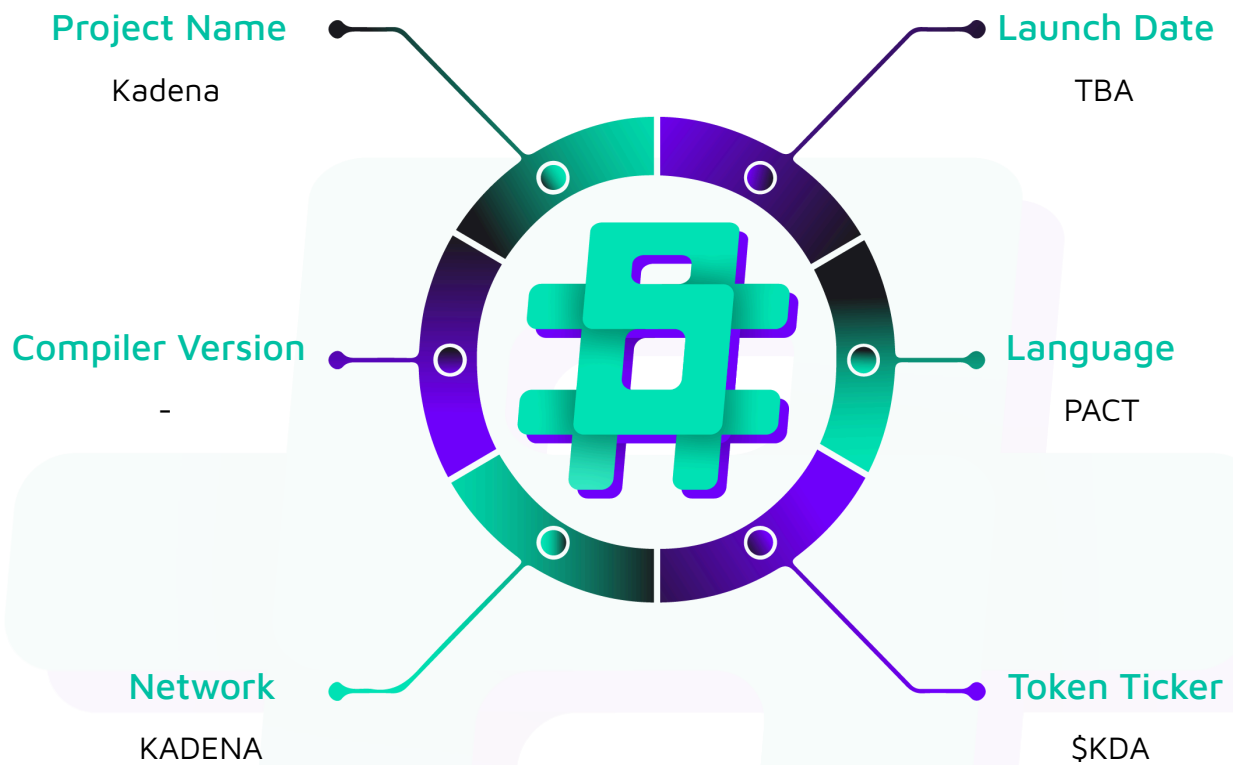
Compiler Version: Pact

Website: <https://www.kadena.io/>

Logo:



Hashlock Pty Ltd

Visualised Context:

Project Visuals:

The Blockchain for Business

Kadena is the only blockchain capable of powering global finance.
Build your future on Kadena.

[Learn More](#)

Combining 250 Years of Blockchain and Traditional Financial Experience

Our team has extensive experience bridging the gap between blockchain and real-world business use cases, working with some of the largest and most influential organizations in the world, including J.P. Morgan, Microsoft, Arca, State Street, and the SEC.



Audit Scope

We at Hashlock audited the pact code within the Kadena project, the scope of work included a comprehensive review of the smart contracts listed below. We tested the smart contracts to check for their security and efficiency. These tests were undertaken primarily through manual line-by-line analysis and were supported by software-assisted testing.

Description	Kadena Smart Contracts
Platform	Kadena / Pact
Audit Date	April, 2025
Contract 1	compliance-v1.pact
Contract 1 MD5 Hash	81261589ee43e14c37cb23c8d2252b42
Contract 2	real-world-asset-v1.pact
Contract 2 MD5 Hash	19c82e1b5fb131291d0e2148c659ead7
Contract 3	claim-topics-registry-v1.pact
Contract 3 MD5 Hash	c1f2da4b356f920fe74b7388f399342f
Contract 4	identity-registry-storage-v1.pact
Contract 4 MD5 Hash	daae92167d4552fd5e634b693c651d79
Contract 5	identity-registry-v1.pact
Contract 5 MD5 Hash	652d4274d2667ff4d2611d913d5f9452
Contract 6	trusted-issuers-registry-v1.pact
Contract 6 MD5 Hash	2d8eb2cc678b057ab9351b422f95d3f9
Contract 7	agent-role-v1.pact
Contract 7 MD5 Hash	4b0b7b8d6c095228dd12041dd7a06235
Contract 8	compliance-compatible-v1.pact
Contract 8 MD5 Hash	09a46cbe2deeb22a5bd9c0a6ff1e3268
Contract 9	max-balance-compliance.pact

Contract 9 MD5 Hash	fc8619244828bfcee97eb0cf189704d6
Contract 10	max-investors-compliance.pact
Contract 10 MD5 Hash	ff6efb7666653ce02abc2957282f6885
Contract 11	supply-limit-compliance.pact
Contract 11 MD5 Hash	e5b027912dcde4a4d3d27b434ccc089a
Contract 12	mvp-token.pact
Contract 12 MD5 Hash	ab7a6b9b779e6b51aea8803c3dae7c3c
Contract 13	token-mapper.pact
Contract 13 MD5 Hash	b872ebd2dad93971262bdd678cefe40e
Originally Audited GitHub Commit Hash	198703133634e82b54849d31144f24e17bc801fe

Security Rating

After Hashlock's Audit, we found the smart contracts to be **"Secure"**. The contracts all follow simple logic, with correct and detailed ordering. We have identified some vulnerabilities that need to be addressed prior to launch.



Not Secure

Vulnerable

Secure

Hashlocked

The 'Hashlocked' rating is reserved for projects that ensure ongoing security via bug bounty programs or on chain monitoring technology.

All issues uncovered during automated and manual analysis were meticulously reviewed and applicable vulnerabilities are presented in the [Audit Findings](#) section. The list of audited assets is presented in the [Audit Scope](#) section and the project's contract functionality is presented in the [Intended Smart Contract Functions](#) section.

All vulnerabilities initially identified have now been resolved and acknowledged.

Hashlock found:

4 High severity vulnerabilities

4 Medium severity vulnerabilities

9 Low severity vulnerabilities

Caution: Hashlock's audits do not guarantee a project's success or ethics, and are not liable or responsible for security. Always conduct independent research about any project before interacting.

Intended Smart Contract Functions

Claimed Behaviour	Actual Behaviour
compliance-v1.pact - Interface defining the compliance rules and behavior for RWA tokens.	Contract achieves this functionality.
real-world-asset-v1.pact - Interface that defines core RWA token functionality.	Contract achieves this functionality.
claim-topic-registry-v1.pact - Interface that defines functionality for managing trusted claim topics in the RWA token ecosystem.	Contract achieves this functionality.
identity-registry-storage-v1.pact - Interface that defines the required events that must be emitted during identity storage operations in the RWA token ecosystem.	Contract achieves this functionality.
identity-registry-v1.pact - Interface that defines core identity management functionality for investor verification and KYC compliance in the RWA token ecosystem.	Contract achieves this functionality.
trusted-issuers-registry-v1.pact - Interface that defines functionality for managing and verifying trusted claim issuers in the RWA token ecosystem. This registry helps maintain a list of authorized entities that can issue identity claims for KYC/AML compliance.	Contract achieves this functionality.
agent-role-v1.pact - Interface that defines role-based access control	Contract achieves this functionality.

functionality for managing agents and ownership in the RWA token ecosystem.	
compliance-compatible-v1.pact - Interface that defines the parameters and functions required for implementing three key compliance rules in RWA tokens: - Maximum balance per investor - Maximum total supply - Maximum number of investors	Contract achieves this functionality.
max-balance-compliance.pact - Contract that implements maximum balance per investor compliance rule for RWA tokens. - Allow admins to: - Bind/unbind tokens to compliance contract - Set maximum balance per investor limits	Contract achieves this functionality.
max-investors-compliance.pact - Contract that implements maximum investor count compliance rule for RWA tokens.	Contract achieves this functionality.
supply-limit-compliance.pact - Compliance contract that enforces maximum supply limits for RWA tokens	Contract achieves this functionality.
mvp-token.pact - Main implementation of the RWA (Real World Asset) token standard with compliance, identity management, and role-based access control. - Allows users to: - Transfer tokens to other verified users - View their token balance and account details - Check if their address is frozen	Contract achieves this functionality.

- Check token compliance status - View token information (name, symbol, etc.) - Recover lost wallets - Allows admins to: - Mint and burn tokens - Freeze/unfreeze accounts - Set token parameters (name, symbol, kadenaID) - Manage compliance rules - Register and manage user identities - Perform batch operations (transfers, mints, burns) - Update supply limits - Pause/unpause contract - Manage agent roles and permissions - Set maximum investor counts - Control maximum balance per investor	
token-mapper.pact - Contract that maps token IDs to their corresponding RWA token implementations, serving as a registry for token contracts - Allows admins to: - Register new token references - Map token IDs to contract implementations - Manage token registry entries	Contract achieves this functionality.

Code Quality

This audit scope involves the smart contracts of the Kadena project, as outlined in the Audit Scope section. All contracts, libraries, and interfaces mostly follow standard best practices and to help avoid unnecessary complexity that increases the likelihood of exploitation, however, some refactoring was required.

The code is very well commented on and closely follows best practice nat-spec styling. All comments are correctly aligned with code functionality.

Audit Resources

We were given the Kadena project smart contract code in the form of Github access.

As mentioned above, code parts are well commented. The logic is straightforward, and therefore it is easy to quickly comprehend the programming flow as well as the complex code logic. The comments are helpful in providing an understanding of the protocol's overall architecture.

Severity Definitions

The severity levels assigned to findings represent a comprehensive evaluation of both their potential impact and the likelihood of occurrence within the system. These categorizations are established based on Hashlock's professional standards and expertise, incorporating both industry best practices and our discretion as security auditors. This ensures a tailored assessment that reflects the specific context and risk profile of each finding.

Significance	Description
High	High-severity vulnerabilities can result in loss of funds, asset loss, access denial, and other critical issues that will result in the direct loss of funds and control by the owners and community.
Medium	Medium-level difficulties should be solved before deployment, but won't result in loss of funds.
Low	Low-level vulnerabilities are areas that lack best practices that may cause small complications in the future.
Gas	Gas Optimisations, issues, and inefficiencies.
QA	Quality Assurance (QA) findings are informational and don't impact functionality. Supports clients improve the clarity, maintainability, or overall structure of the code.

Status Definitions

Each identified security finding is assigned a status that reflects its current stage of remediation or acknowledgment. The status provides clarity on the handling of the issue and ensures transparency in the auditing process. The statuses are as follows:

Significance	Description
Resolved	The identified vulnerability has been fully mitigated either through the implementation of the recommended solution proposed by Hashlock or through an alternative client-provided solution that demonstrably addresses the issue.
Acknowledged	The client has formally recognized the vulnerability but has chosen not to address it due to the high cost or complexity of remediation. This status is acceptable for medium and low-severity findings after internal review and agreement. However, all high-severity findings must be resolved without exception.
Unresolved	The finding remains neither remediated nor formally acknowledged by the client, leaving the vulnerability unaddressed.

Audit Findings

High

[H-01] mvp-token#transfer-internal - Function does not check if the receiving address is frozen

Description

The `transfer-internal` function lacks the necessary check to see if the receiver address is frozen. This allows tokens to be sent to frozen addresses that should not be able to receive or send tokens. The indicated is against the specifications that are defined in the ERC-3643 standard.

Vulnerability Details

The `transfer-internal` function handles token transfers

```
(defun transfer-internal:string
  ( sender:string
    receiver:string
    amount:decimal
  )

  @doc "Transfer AMOUNT of token from sender to receiver. Validates can-transfer rules
in compliance"

  (require-capability (TRANSFER sender receiver amount))

  (enforce-unfrozen sender)

  (enforce-unfrozen-amount sender amount)

  (enforce-contains-identity receiver)

  (with-read token DEFAULT-ROW {

    "compliance" := compliance-1
```



```

    , "paused" := paused
  }

  (enforce (= paused false) "TRF-PAUSE-001")

  (map
    (lambda (compliance:module {RWA.compliance-v1})
      (compliance::can-transfer TOKEN-ID sender receiver amount)
    ) compliance-l
  )

  (let
    ( (sender-obj (debit sender amount))
      (receiver-obj (credit receiver amount)) )
    (emit-event (RECONCILE amount sender-obj receiver-obj))

    (map
      (lambda (compliance:module {RWA.compliance-v1})
        (compliance::transferred TOKEN-ID sender receiver amount)
      ) compliance-l
    )
  )

  "Transfer successful"
)

```

As seen in the highlighted parts, the enforcements only check if:

- Sender is not frozen
- Sender has enough unfrozen token balance
- Receiver is KYCd

Taking a look at the ERC3643 standard written in Solidity



```

function transfer(address _to, uint256 _amount) public override whenNotPaused returns
(bool) {
    require(!_frozen[_to] && !_frozen[msg.sender], "wallet is frozen");
    require(_amount <= balanceOf(msg.sender) - (_frozenTokens[msg.sender]),
    "Insufficient Balance");
    if (_tokenIdRegistry.isVerified(_to) &&
_tokenCompliance.canTransfer(msg.sender, _to, _amount)) {
        _transfer(msg.sender, _to, _amount);
        _tokenCompliance.transferred(msg.sender, _to, _amount);
        return true;
    }
    revert("Transfer not possible");
}

```

It is observed that the receiver address should be unfrozen for a transfer to succeed. This means that the version written in Pact does not follow the ERC3643 rule for token transfer, and frozen addresses are able to receive tokens. These tokens sent to frozen addresses would be stuck in that address until they are unfrozen.

Proof of Concept

An example of a test that simulates the vulnerability is shown below.

```

(load "../contracts/test/install-contracts.repl")

(begin-tx "Initialize contracts")

(env-data { 'ns: "RWA" })

(load "../token-mapper.pact")

(load "../compliances/compliance-compatible-v1.pact")

(load "../mvp-token.pact")

(load "../compliances/max-balance-compliance.pact")

(load "../compliances/max-investors-compliance.pact")

(load "../compliances/supply-limit-compliance.pact")

```

```

;; Set up admin keyset

(env-data {

  "admin-keyset": {"keys": ["admin"], "pred": "keys-all"}

})

(define-keyset "RWA.rwa-admin-keyset" (read-keyset 'admin-keyset))


;; Initialize token

(env-data {"ns": "RWA", "owner-guard":{"pred": "keys-all", "keys": ["owner"]}})

(env-sigs [{ 'key: 'admin, 'caps: [(RWA.mvp-token.GOV)]})

(RWA.mvp-token.init "mvp-token" "MVP" 0 "kadenaID"

  [RWA.max-balance-compliance RWA.supply-limit-compliance RWA.max-investors-compliance]

  false (read-keyset "owner-guard"))

(commit-tx)


;; Add agent

(begin-tx "Add agent")

(env-data {"agent-guard":{"pred": "keys-all", "keys": ["agent"]},

  "roles": ["freezer", "agent-admin", "transfer-manager"]})

(env-sigs [

  { 'key: 'owner

    , 'caps: [(RWA.mvp-token.ONLY-AGENT "owner")]

  }])

(RWA.mvp-token.add-agent 'agent1 (read-keyset 'agent-guard))

(commit-tx)


;; Register first identity

```

```

(begin-tx "Register first identity")

(env-data {

  "investor-a": {"keys": ["investor-a"], "pred": "keys-all"}

})

(env-sigs [

  { 'key: 'owner

    , 'caps: [(RWA.mvp-token.ONLY-AGENT "owner")

              (RWA.mvp-token.INTERNAL)]}

])

(RWA.mvp-token.register-identity "investor-a-address" (read-keyset "investor-a") "" 1)

(commit-tx)

;; Register second identity

(begin-tx "Register second identity")

(env-data {

  "investor0": {"keys": ["investor0"], "pred": "keys-all"}

})

(env-sigs [

  { 'key: 'owner

    , 'caps: [(RWA.mvp-token.ONLY-AGENT "owner")

              (RWA.mvp-token.INTERNAL)]}

])

(RWA.mvp-token.register-identity "investor0" (read-keyset "investor0") "" 1)

(commit-tx)

;; Mint initial balances

```

```

(begin-tx "Mint for investor-a-address")

(env-data {
  "agent": "agent1"
})

(env-sigs [
  { 'key: 'agent
    , 'caps: [(RWA.mvp-token.ONLY-AGENT "transfer-manager")
              (RWA.mvp-token.TRANSFER "" "investor-a-address" 10.0)
              (RWA.mvp-token.MINT)]}
])

(RWA.mvp-token.mint "investor-a-address" 10.0)

(commit-tx)

(begin-tx "Mint for investor0")

(env-data {
  "agent": "agent1"
})

(env-sigs [
  { 'key: 'agent
    , 'caps: [(RWA.mvp-token.ONLY-AGENT "transfer-manager")
              (RWA.mvp-token.TRANSFER "" "investor0" 9.0)
              (RWA.mvp-token.MINT)]}
])

(RWA.mvp-token.mint "investor0" 9.0)

(commit-tx)

```

```

;; Freeze receiver account

(begin-tx "Freeze investor0")

(env-data {

  "agent": "agent1"

})

(env-sigs [

  { 'key: 'agent

    , 'caps: [(RWA.mvp-token.ONLY-AGENT "freezer")]

  }])

(RWA.mvp-token.set-address-frozen "investor0" true)

;; Verify state

(expect

  "Verify investor0 is frozen"

  true

  (RWA.mvp-token.address-frozen "investor0"))

(commit-tx)

(begin-tx "Test transfers to frozen accounts")

;; Verify initial state: account is frozen and balances

(expect "Verify investor0 is frozen"

  true

  (RWA.mvp-token.address-frozen "investor0"))

```

```

(expect "Initial balances correct"

  [10.0 9.0]

  [(RWA.mvp-token.get-balance "investor-a-address")

   (RWA.mvp-token.get-balance "investor0")])

;; Perform transfer to frozen account

(env-sigs [

  { 'key: 'investor-a

    , 'caps: [(RWA.mvp-token.TRANSFER "investor-a-address" "investor0" 1.0)]

  }])

;; Execute transfer - this succeeds despite receiver being frozen

(RWA.mvp-token.transfer "investor-a-address" "investor0" 1.0)

;; Verify balances changed despite frozen receiver

(expect "Balances changed despite frozen receiver"

  [9.0 10.0]

  [(RWA.mvp-token.get-balance "investor-a-address")

   (RWA.mvp-token.get-balance "investor0")])

;; Verify receiver is still frozen

(expect "Receiver still frozen after transfer"

  true

  (RWA.mvp-token.address-frozen "investor0"))

(commit-tx)

```

Impact

It is possible to transfer tokens to frozen addresses. These tokens that are sent will be stuck in the frozen addresses.

Recommendation

Implement a check to make sure that the receiving address is not frozen.

Status

Resolved

[H-02] mvp-token#recovery-address - It is impossible for users that lost access to their private keys to recover their tokens

Description

The recovery-address function is used when a user loses access to their private keys. This function forcefully transfers the tokens in the lost address to a recovery address. However, this function is not properly implemented and recovery is impossible.

Vulnerability Details

The recovery-address function is used when a user loses access to their private keys

```
(defun recovery-address:bool (lost-wallet:string new-wallet:string
investor-kadenaID:string)

  @doc "Recover tokens from a lost wallet to a new wallet."

  (only-agent AGENT-ADMIN)

  (emit-event (RECOVERY-SUCCESS lost-wallet new-wallet investor-kadenaID))

  false

)
```

As observed in this function, it does not have any functionality other than emitting an event.

The specifications for ERC3643 state that the contract must have a recovery system in case an investor loses access to their private key. Taking a look at the Solidity version, an example of this can be seen.

```
function recoveryAddress(
    address _lostWallet,
    address _newWallet,
    address _investorOnchainID
) external override onlyAgent returns (bool) {
    require(balanceOf(_lostWallet) != 0, "no tokens to recover");

    IIdentity _onchainID = IIdentity(_investorOnchainID);
    bytes32 _key = keccak256(abi.encode(_newWallet));

    if (_onchainID.keyHasPurpose(_key, 1)) {
        uint256 investorTokens = balanceOf(_lostWallet);
        uint256 frozenTokens = _frozenTokens[_lostWallet];

        _tokenIdRegistry.registerIdentity(_newWallet, _onchainID,
            _tokenIdRegistry.investorCountry
                (_lostWallet));

        forcedTransfer(_lostWallet, _newWallet, investorTokens);

        if (frozenTokens > 0) {
            freezePartialTokens(_newWallet, frozenTokens);
        }

        if (_frozen[_lostWallet] == true) {
            setAddressFrozen(_newWallet, true);
        }

        _tokenIdRegistry.deleteIdentity(_lostWallet);

        emit RecoverySuccess(_lostWallet, _newWallet, _investorOnchainID);

        return true;
    }
}
```

```
}  
  
    revert("Recovery not possible");  
  
}
```

Impact

Users who lose access to their private keys will not be able to recover their tokens.

Recommendation

Implement the token recovery functionality.

Status

Resolved

[H-03] mvp-token#forced-transfer - It is impossible to forcefully transfer out of frozen addresses

Description

The forced-transfer function allows the privileged role `TRANSFER-MANAGER` to forcefully transfer tokens from one address to another address. This functionality can be used when:

- A user has lost access to their private key and requires these tokens to be transferred to their new address
- A user has had their private key stolen
- A user is acting maliciously and faces token loss as punishment

The situations listed above would need the first address to be frozen in order to stop potential losses. However, in the Pact implementation, it is impossible to forcefully transfer tokens out of frozen addresses, which contradicts the Solidity implementation.

Vulnerability Details

The forced-transfer function allows the privileged role `TRANSFER-MANAGER` to forcefully transfer tokens from one address to another address:

```
(defun forced-transfer:bool (from:string to:string amount:decimal)

  @doc "Force transfer tokens from one address to another."

  (only-agent TRANSFER-MANAGER)

  (with-capability (FORCED-TRANSFER)

    (with-capability (TRANSFER from to amount)

      (transfer-internal from to amount)

    ))

  true

)
```

As observed, this function calls the `transfer-internal` function that is used for normal transfers and employs restrictions for frozen accounts:

```
(defun transfer-internal:string
  ( sender:string
    receiver:string
    amount:decimal
  )
  @doc "Transfer AMOUNT of token from sender to receiver. Validates can-transfer rules
in compliance"
  (require-capability (TRANSFER sender receiver amount))
  (enforce-unfrozen sender)
  (enforce-unfrozen-amount sender amount)
  (enforce-contains-identity receiver)
  (with-read token DEFAULT-ROW {
    "compliance" := compliance-l
    , "paused" := paused
  })
  (enforce (= paused false) "TRF-PAUSE-001")
  (map
    (lambda (compliance:module {RWA.compliance-v1})
      (compliance::can-transfer TOKEN-ID sender receiver amount)
    ) compliance-l
  )
  (let
```

```

    ( (sender-obj (debit sender amount))
      (receiver-obj (credit receiver amount)) )
    (emit-event (RECONCILE amount sender-obj receiver-obj))
    (map
      (lambda (compliance:module{RWA.compliance-v1})
        (compliance::transferred TOKEN-ID sender receiver amount)
      ) compliance-l
    )
  )
)
)
"Transfer successful"
)

```

This means that when an address is frozen for reasons such as acting maliciously or an address being hacked, it will be impossible to forcefully transfer tokens out of this address. As a prevention, privileged roles can unfreeze this address before a forced transfer, but this must be done atomically, as the malicious actors can sandwich their transactions between these transactions to avoid forced transfer of their tokens.

Taking a look at the Solidity implementation:

```

function forcedTransfer(
    address _from,
    address _to,
    uint256 _amount
) public override onlyAgent returns (bool) {
    require(balanceOf(_from) >= _amount, "sender balance too low");
    uint256 freeBalance = balanceOf(_from) - (_frozenTokens[_from]);
    if (_amount > freeBalance) {
        uint256 tokensToUnfreeze = _amount - (freeBalance);
        _frozenTokens[_from] = _frozenTokens[_from] - (tokensToUnfreeze);
        emit TokensUnfrozen(_from, tokensToUnfreeze);
    }
    if (_tokenIdRegistry.isVerified(_to)) {
        _transfer(_from, _to, _amount);
        _tokenCompliance.transferred(_from, _to, _amount);
        return true;
    }
    revert("Transfer not possible");
}

```

```
}
```

It is observed that the regular transfer restrictions are not applied to forced transfers.

Proof of Concept

An example of a test that simulates the vulnerability is shown below:

```
;; Load contracts
(load "../contracts/test/install-contracts.repl")

(begin-tx "Initialize contracts")
(env-data { 'ns: "RWA"})
(load "./token-mapper.pact")
(load "./compliances/compliance-compatible-v1.pact")
(load "./mvp-token.pact")
(load "./compliances/max-balance-compliance.pact")
(load "./compliances/max-investors-compliance.pact")
(load "./compliances/supply-limit-compliance.pact")

(env-data {
  "admin-keyset": {"keys": ["admin"], "pred": "keys-all"}
})
(define-keyset "RWA.rwa-admin-keyset" (read-keyset 'admin-keyset))

(env-data {"ns": "RWA", "owner-guard":{"pred": "keys-all", "keys": ["owner"]}})
(env-sigs [{ 'key: 'admin, 'caps: [(RWA.mvp-token.GOV)]}])
(RWA.mvp-token.init "mvp-token" "MVP" 0 "kadenaID"
  [RWA.max-balance-compliance RWA.supply-limit-compliance RWA.max-investors-compliance]
  false (read-keyset "owner-guard"))
(commit-tx)

(begin-tx "Add transfer manager")
(env-data {
  "agent-guard": {"pred": "keys-all", "keys": ["manager"]},
  "roles": ["transfer-manager"],
  "agent": "manager"
})
(env-sigs [
```

```

{ 'key: 'owner
  , 'caps: [(RWA.mvp-token.ONLY-AGENT "owner")]}
])
(RWA.mvp-token.add-agent "manager" (read-keyset "agent-guard"))
(commit-tx)

(begin-tx "Add freezer agent")
(env-data {
  "agent-guard": {"pred": "keys-all", "keys": ["freezer"]},
  "roles": ["freezer"],
  "agent": "freezer"
})
(env-sigs [
  { 'key: 'owner
    , 'caps: [(RWA.mvp-token.ONLY-AGENT "owner")]}
])
(RWA.mvp-token.add-agent "freezer" (read-keyset "agent-guard"))
(commit-tx)

(begin-tx "Register Alice identity")
(env-data {
  "sender": {"keys": ["alice"], "pred": "keys-all"}
})
(env-sigs [
  { 'key: 'owner
    , 'caps: [(RWA.mvp-token.ONLY-AGENT "owner")
              (RWA.mvp-token.INTERNAL)]}
])
(RWA.mvp-token.register-identity "alice" (read-keyset "sender") "ALICE-ID-001" 1)
(commit-tx)

(begin-tx "Register Bob identity")
(env-data {
  "receiver": {"keys": ["bob"], "pred": "keys-all"}
})
(env-sigs [
  { 'key: 'owner
    , 'caps: [(RWA.mvp-token.ONLY-AGENT "owner")
              (RWA.mvp-token.INTERNAL)]}
])

```

```

])
(RWA.mvp-token.register-identity "bob" (read-keyset "receiver") "BOB-ID-001" 1)
(commit-tx)

(begin-tx "Mint tokens to Alice")
(env-data {
  "agent": "manager"
})
(env-sigs [
  { 'key: 'manager
    , 'caps: [(RWA.mvp-token.ONLY-AGENT "transfer-manager")
              (RWA.mvp-token.TRANSFER "" "alice" 1000.0)
              (RWA.mvp-token.MINT)]}]
])
(RWA.mvp-token.mint "alice" 1000.0)
(commit-tx)

(begin-tx "Verify initial balances")
(use RWA.mvp-token)
(expect "Alice should have 1000 tokens" 1000.0 (get-balance "alice"))
(expect "Bob should have 0 tokens" 0.0 (get-balance "bob"))
(commit-tx)

;; Freeze Alice's account
(begin-tx "Freeze Alice's account")
(env-data {
  "agent": "freezer"
})
(env-sigs [
  { 'key: 'freezer
    , 'caps: [(RWA.mvp-token.ONLY-AGENT "freezer")]}
])
(RWA.mvp-token.set-address-frozen "alice" true)
(commit-tx)

;; Try normal transfer from frozen account
(begin-tx "Try normal transfer from frozen account")
(env-sigs [
  { 'key: 'alice

```

```

    , 'caps: [(RWA.mvp-token.TRANSFER "alice" "bob" 500.0)]})
  ])
  (expect-failure "Regular transfers from frozen accounts should fail"
    "ACC-FRZ-001"
    (RWA.mvp-token.transfer "alice" "bob" 500.0))
  (commit-tx)

;; Try forced transfer from frozen account
(begin-tx "Try forced transfer from frozen account")
(env-data {
  "agent": "manager"
})
(env-sigs [
  { 'key: 'manager
    , 'caps: [(RWA.mvp-token.ONLY-AGENT "transfer-manager")
              (RWA.mvp-token.TRANSFER "alice" "bob" 500.0)
              (RWA.mvp-token.FORCED-TRANSFER)]})
])
(expect-failure "Even forced transfers can't move tokens from frozen accounts"
  "ACC-FRZ-001"
  (RWA.mvp-token.forced-transfer "alice" "bob" 500.0))

(expect "Alice still has 1000 tokens" 1000.0 (RWA.mvp-token.get-balance "alice"))
(expect "Bob still has 0 tokens" 0.0 (RWA.mvp-token.get-balance "bob"))
(commit-tx)

;; Unfreeze Alice to demonstrate that forced transfer works when not frozen
(begin-tx "Unfreeze Alice's account")
(env-data {
  "agent": "freezer"
})
(env-sigs [
  { 'key: 'freezer
    , 'caps: [(RWA.mvp-token.ONLY-AGENT "freezer")]})
])
(RWA.mvp-token.set-address-frozen "alice" false)
(commit-tx)

(begin-tx "Execute forced transfer now that account is unfrozen")

```



```
(env-data {
  "agent": "manager"
})
(env-sigs [
  { 'key: 'manager
    , 'caps: [(RWA.mvp-token.ONLY-AGENT "transfer-manager")
              (RWA.mvp-token.TRANSFER "alice" "bob" 500.0)
              (RWA.mvp-token.FORCED-TRANSFER)]}
])
(RWA.mvp-token.forced-transfer "alice" "bob" 500.0)
(expect "Alice now has 500 tokens" 500.0 (RWA.mvp-token.get-balance "alice"))
(expect "Bob now has 500 tokens" 500.0 (RWA.mvp-token.get-balance "bob"))
(commit-tx)
```

Impact

The Pact implementation does not match the Solidity implementation. It will not be possible to forcefully transfer tokens out of frozen addresses. Unfreezing an address before forced transfer runs the risk of malicious actors sandwiching their transaction between these two calls to avoid the forced transfer.

Recommendation

Implement a new `forced-transfer-internal` function to be used by `forced-transfer` that will bypass restrictions applied to token transfer.

Status

Resolved

[H-04] mvp-token#forced-transfer - Function does not unfreeze required amount of tokens for the forced transfer to succeed

Description

The `forced-transfer` function allows the privileged role `TRANSFER-MANAGER` to forcefully transfer tokens from one address to another address. In the Solidity implementation, if the amount of tokens to be forcefully transferred is partially frozen in a user's balance,

the required amount is unfrozen before the transfer. However, Pact implementation does not have this functionality.

Vulnerability Details

The `forced-transfer` function allows the privileged role `TRANSFER-MANAGER` to forcefully transfer tokens from one address to another address:

```
(defun forced-transfer:bool (from:string to:string amount:decimal)

  @doc "Force transfer tokens from one address to another."

  (only-agent TRANSFER-MANAGER)

  (with-capability (FORCED-TRANSFER)

    (with-capability (TRANSFER from to amount)

      (transfer-internal from to amount)

    ))

  true

)
```

Compared to the Solidity version, this function does not unfreeze the required amount of frozen tokens from the sender address:

```
function forcedTransfer(
    address _from,
    address _to,
    uint256 _amount
) public override onlyAgent returns (bool) {
    require(balanceOf(_from) >= _amount, "sender balance too low");
    uint256 freeBalance = balanceOf(_from) - (_frozenTokens[_from]);
    if (_amount > freeBalance) {
        uint256 tokensToUnfreeze = _amount - (freeBalance);
        _frozenTokens[_from] = _frozenTokens[_from] - (tokensToUnfreeze);
        emit TokensUnfrozen(_from, tokensToUnfreeze);
    }
    if (_tokenIdRegistry.isVerified(_to)) {
        _transfer(_from, _to, _amount);
    }
}
```

```
        _tokenCompliance.transferred(_from, _to, _amount);  
        return true;  
    }  
    revert("Transfer not possible");  
}
```

As a result, if the sender address does not have enough unfrozen balance, the forced transfer will not be possible. The only way to make this transfer possible would be to first unfreeze the amount required from the sender and then forcefully transfer these tokens. However, malicious actors can sandwich their transactions between these two transactions to avoid forced token transfer out of their address.

Impact

The Pact implementation does not match the Solidity implementation. It will not be possible to forcefully transfer tokens from addresses that do not have enough unfrozen balance. Unfreezing tokens from an address before forced transfer runs the risk of malicious actors sandwiching their transaction between these two calls to avoid the forced transfer.

Recommendation

Implement the functionality in `forced-transfer` that automatically unfreezes a user's tokens atomically if they do not have enough unfrozen balance.

Status

Resolved

Medium

[M-01] mvp-token#unfreeze-partial-tokens-internal - The function emits the wrong event which will lead to confusion in off-chain systems and listeners

Description

The `unfreeze-partial-tokens-internal` function is responsible for unfreezing user tokens. However, this function emits the wrong event, which will lead to confusion for listeners and might lead to off-chain systems performing incorrect actions.

Vulnerability Details

The `unfreeze-partial-tokens-internal` function is responsible for unfreezing user tokens:

```
(defun unfreeze-partial-tokens-internal:string (investor-address:string amount:decimal)

  @doc "Internal function to update the partial freeze."

  (require-capability (INTERNAL))

  (enforce (> amount ZERO-DECIMAL) "FRZ-AMT-004")

  (enforce-unit amount)

  (with-read investors investor-address {

    "balance" := balance,

    "frozen" := frozen,

    "amount-frozen" := amount-frozen

  })

  (enforce (not frozen) "ACC-FRZ-001")

  (enforce (<= amount amount-frozen) "FRZ-AMT-003")

  (update investors investor-address {"amount-frozen": (- amount-frozen amount)})

  (emit-event (TOKENS-FROZEN investor-address (- amount-frozen amount)))

  "Partial tokens unfrozen"
```

```
)
)
```

As observed, this function emits the `TOKENS-FROZEN` event instead of the `TOKENS-UNFROZEN` event. When a user's tokens are unfrozen, the function will incorrectly emit an event as if this user's tokens are being frozen. As seen in the ERC3643 token contract written in Solidity, there is a different event for unfreezing tokens:

```
function unfreezePartialTokens(address _userAddress, uint256 _amount) public override
onlyAgent {
    require(_frozenTokens[_userAddress] >= _amount, "Amount should be less than or
equal to frozen tokens");
    _frozenTokens[_userAddress] = _frozenTokens[_userAddress] - (_amount);
    emit TokensUnfrozen(_userAddress, _amount);
}
```

Impact

This vulnerability will cause confusion for listeners, and it can lead to off-chain systems performing incorrect actions.

Recommendation

Update the function to emit the correct event.

Status

Resolved

[M-02] `mvp-token#freeze-partial-tokens-internal, unfreeze-partial-tokens-internal` - Functions emit the wrong amount as frozen/unfrozen

Description

The `freeze-partial-tokens-internal` and `unfreeze-partial-tokens-internal` functions are responsible for freezing/unfreezing a user's tokens. However, these functions use

the wrong value when emitting an event for the amount of tokens that are frozen/unfrozen.

Vulnerability Details

The `freeze-partial-tokens-internal` and `unfreeze-partial-tokens-internal` functions are responsible for freezing/unfreezing a user's tokens:

```
(defun freeze-partial-tokens-internal:string (investor-address:string amount:decimal)

  @doc "Internal function to update the partial freeze."

  (require-capability (INTERNAL))

  (enforce (> amount ZERO-DECIMAL) "FRZ-AMT-004")

  (enforce-unit amount)

  (with-read investors investor-address {

    "balance" := balance,

    "frozen" := frozen,

    "amount-frozen" := amount-frozen

  })

  (enforce (not frozen) "ACC-FRZ-001")

  (enforce (>= balance (+ amount-frozen amount)) "FRZ-AMT-002")

  (update investors investor-address {"amount-frozen": (+ amount-frozen amount)}))

  (emit-event (TOKENS-FROZEN investor-address (+ amount-frozen amount)))

  "Partial tokens frozen"

)

)

(defun unfreeze-partial-tokens-internal:string (investor-address:string amount:decimal)

  @doc "Internal function to update the partial freeze."

  (require-capability (INTERNAL))
```

```

(enforce (> amount ZERO-DECIMAL) "FRZ-AMT-004")

(enforce-unit amount)

(with-read investors investor-address {

    "balance" := balance,

    "frozen" := frozen,

    "amount-frozen" := amount-frozen

})

(enforce (not frozen) "ACC-FRZ-001")

(enforce (<= amount amount-frozen) "FRZ-AMT-003")

(update investors investor-address {"amount-frozen": (- amount-frozen amount)})

(emit-event (TOKENS-FROZEN investor-address (- amount-frozen amount)))

    "Partial tokens unfrozen"

)

)

```

As observed in these functions, the amount that is used in the events that are being emitted is `amount-frozen + amount` and `amount-frozen - amount`, respectively. However, the amount that was frozen/unfrozen with these functions is only the input amounts. This can also be seen in the ERC3643 token contract written in Solidity:

```

function freezePartialTokens(address _userAddress, uint256 _amount) public override
onlyAgent {

    uint256 balance = balanceOf(_userAddress);

    require(balance >= _frozenTokens[_userAddress] + _amount, "Amount exceeds
available balance");

    _frozenTokens[_userAddress] = _frozenTokens[_userAddress] + (_amount);

    emit TokensFrozen(_userAddress, _amount);

}

```

```
function unfreezePartialTokens(address _userAddress, uint256 _amount) public override
onlyAgent {

    require(_frozenTokens[_userAddress] >= _amount, "Amount should be less than or
equal to frozen tokens");

    _frozenTokens[_userAddress] = _frozenTokens[_userAddress] - (_amount);

    emit TokensUnfrozen(_userAddress, _amount);

}
```

Impact

This vulnerability will cause confusion for listeners, and it can lead to off-chain systems performing incorrect actions.

Recommendation

Update the function to use the correct amount when emitting the events.

Status

Resolved

[M-03] mvp-token#transfer-internal - Function does not check for the returned value from compliance contracts

Description

The `transfer-internal` function relies on the compliance contracts reverting when a token transfer should not happen. However, this is different from the Solidity version of the ERC3643. Not all compliance contracts are guaranteed to revert on failure and they might return booleans as the result.

Vulnerability Details

The `transfer-internal` function relies on the compliance contracts reverting when a token transfer should not happen:

```
(defun transfer-internal:string

  ( sender:string
```




```

    receiver:string

    amount:decimal

)

@doc "Transfer AMOUNT of token from sender to receiver. Validates can-transfer rules
in compliance"

(require-capability (TRANSFER sender receiver amount))

(enforce-unfrozen sender)

(enforce-unfrozen-amount sender amount)

(enforce-contains-identity receiver)

(with-read token DEFAULT-ROW {

    "compliance" := compliance-1

    , "paused" := paused

})

(enforce (= paused false) "TRF-PAUSE-001")

(map

    (lambda (compliance:module {RWA.compliance-v1})

        (compliance::can-transfer TOKEN-ID sender receiver amount)

    ) compliance-1

)

(let

    ( (sender-obj (debit sender amount))

        (receiver-obj (credit receiver amount)) )

    (emit-event (RECONCILE amount sender-obj receiver-obj))

    (map

        (lambda (compliance:module {RWA.compliance-v1})

            (compliance::transferred TOKEN-ID sender receiver amount)

        ) compliance-1

)

```

```

    )

    )

    )

    "Transfer successful"

    )

```

As observed in this function, the returned boolean from the compliance contracts are not checked, and the function relies on the compliance contracts reverting on failure. Taking a look at the ERC3643 contract written in Solidity:

```

function transfer(address _to, uint256 _amount) public override whenNotPaused returns
(bool) {

    require(!_frozen[_to] && !_frozen[msg.sender], "wallet is frozen");

    require(_amount <= balanceOf(msg.sender) - (_frozenTokens[msg.sender]),
    "Insufficient Balance");

    if (_tokenIdentityRegistry.isVerified(_to) &&
    _tokenCompliance.canTransfer(msg.sender, _to, _amount)) {

        _transfer(msg.sender, _to, _amount);

        _tokenCompliance.transferred(msg.sender, _to, _amount);

        return true;

    }

    revert("Transfer not possible");

}

```

In the highlighted part, it is observed that the transfer reverts based on the boolean returned from the compliance contract.

Even though the current compliance contracts follow this pattern, this is a diversion from the Solidity version of ERC3643, and not all compliance contracts might follow this pattern.

Impact

When compliance contracts that do not revert on failure but instead return a boolean to indicate the result are used, the `transfer-internal` function will not revert when it should.

Recommendation

Refactor the function in order to check the returned boolean from compliance contracts.

Status

Acknowledged

[M-04] mvp-token#register-identity-internal - Function does enforce unique kadenaID values

Description

The `register-identity-internal` function is used to register an investor's identity. This function takes an `investor-identity` input to be assigned as the investor's `kadenaID`. However, the function has no checks implemented to make sure that the inputted `investor-identity` is unique per address and it is not used to register a user already. This will lead to issues on systems that will be implemented that will rely on the investor identity.

Vulnerability Details

The `register-identity-internal` function is used to register an investor's identity:

```
(defun register-identity-internal:bool (investor-address:string investor-guard:guard
investor-identity:string country:integer)
  @doc "Register an identity contract corresponding to an investor address.
\
  \ Requires that the investor doesn't have an identity contract already registered.
\
  \ Only a wallet set as agent of the smart contract can call this function.
\
```

```

\ Emits an `IDENTITY-REGISTERED` event."
(require-capability (INTERNAL))
(is-principal investor-address)
(write identities investor-address { "kadenaID":investor-identity, "country":country,
"active":true })
(with-default-read investors investor-address
  { "balance": -1.0 }
  { "balance"::=balance }
  (if (= balance -1.0)
    (create-account investor-address investor-guard)
    "Account already exists"
  )
)
(emit-event (IDENTITY-REGISTERED investor-address investor-guard investor-identity))
)

```

As observed in this function, there are no checks to make sure that the inputted investor-identity value is not enforced to be unique per address, a kadenaID that was previously assigned to an investor can be assigned to a new investor.

Impact

Although the kadenaID is not being utilized at the time of the audit, Hashlock has been informed about plans on using this value. Situations where different investors will have the same identity will cause issues for the systems that implement it.

Recommendation

Enforce kadenaID to be unique by adding checks in the function.

Status

Acknowledged

Low

[L-01] mvp-token - Prevent updating state variables with same values

Description

Not only is it wasteful in terms of gas, but this is especially problematic when an event is emitted and the old and new values set are the same, as listeners might not expect this kind of scenario. Instances are shown below:

```
(defun set-address-frozen-internal:string (investor-address:string freeze:bool)

(defun delete-identity:bool (investor-address:string)

(defun set-name:string (name:string)

(defun transfer-ownership:bool (new-owner-guard:guard)

(defun set-max-balance-per-investor:bool (max-balance-per-investor:decimal)

(defun set-supply-limit:bool (supply-limit:decimal)

(defun set-max-investors:bool (max-investors:integer)

(defun set-compliance:string (compliance:[module{RWA.compliance-v1}])

(defun update-agent-roles:bool (agent:string roles:[string])
```

Recommendation

Implement checks in the functions to prevent this scenario. An example is shown below:

```
(defun set-name:string (name:string)
  @doc "Set the name of the token."
  (only-agent OWNER)
  (with-read token DEFAULT-ROW {
    "name" := current-name,
    "symbol" := symbol,
    "version" := version,
    "kadenaID" := kadenaID
  })
  (enforce (≠ current-name name) "Name is already set to this value")
  (write token DEFAULT-ROW {"name": name})
```

```
(emit-event (UPDATED-TOKEN-INFORMATION name symbol MINIMUM-PRECISION VERSION
kadenaID ))
)
)
)
```

Status

Resolved

[L-02] mvp-token#mint-internal - It is possible to mint tokens to frozen accounts

Description

There are no checks to make sure that an account is not frozen while attempting to mint tokens to this address. Frozen accounts can not make use of the minted tokens and in order to prevent potential mistakes, it is good practice to have checks against this scenario occurring.

Recommendation

Implement a check in the function to prevent tokens from being minted to frozen addresses.

Status

Resolved

[L-03] mvp-token - Lack of a single function to pre-check if a transfer is going to pass or fail

Description

The ERC3643 specification mentions that one of the requirements is the contract must have a standard interface to pre-check if a transfer is going to pass or fail before sending it to the blockchain. There is no one function that combines the result of all compliance contracts. Users need to manually call the `can-transfer` function in every compliance contract in order to know if their transfer will pass or fail.

Recommendation

Implement a new function that allows users to easily pre-check the result of a transfer by combining all compliance contracts.

Status

Acknowledged

[L-04] mvp-token#transfer-ownership - Two-step ownership transfer should be used

Description

It is good practice to implement a two-step ownership transfer instead of a direct transfer to prevent potential mistakes.

Recommendation

Refactor the function to implement a two-step ownership transfer that requires the new owner to accept the role.

Status

Acknowledged

[L-05] mvp-token#set-max-balance-per-investor, set-supply-limit, set-max-investors - Limits can be updated to values that are lower than existing values that leads to an inconsistent contract state

Description

The setter functions set the limit for token compliance. However, there are no checks in these setter functions to make sure that new limits are not lower than already existing amounts. For example, in a situation with x amount of active registered investors, the limit for maximum allowed investors can be set to a value that is lower than x. This will lead to an inconsistent contract state.

Recommendation

Implement checks in the setter functions to prevent the explained scenario.

Status

Resolved

[L-06] `mvp-token#init, set-compliance` - Lack of checks for duplicate inputs

Description

These functions allow the privileged role to set the compliance modules for the token. However, there are no checks implemented to make sure that the inputted compliance modules are not repeated in the inputs. This will lead to wasteful checks being done on operations and potentially not implementing critical compliances.

Recommendation

Implement checks to make sure that the inputted compliance modules are unique.

Status

Resolved

[L-07] `mvp-token#register-identity, register-identity-internal` - Misleading event emission and comments

Description

The functions specified in the comments that:

```
\ Requires that the investor doesn't have an identity contract already registered.
```

However, this is not enforced in code and this function can be used to update an already existing user's identity. This situation will also lead to incorrect event emission `IDENTITY-REGISTERED` even though no new identity is registered.

Recommendation

Correctly enforce the requirement that users with existing identity modules can not use with this function.

Additionally, implement the unused functions that are meant for identity updates.

Status

Resolved

[L-08] mvp-token#Batch functions - Batch functions do not require array lengths to be matching

Description

Not requiring the input arrays to be of the same length will lead to input mistakes that will cause unintended results or unintended reverts in the functions that make use of more than one array as an input.

Recommendation

Enforce input arrays to have the same length.

Status

Resolved

[L-09] mvp-token#init - Function does not make use of the inputted paused boolean

Description

The `init` function receives a paused input but does not make use of this input and instead sets the paused status to false by default.

Recommendation

Change the function to make use of the inputted pause boolean

Status

Resolved



Centralisation

The Kadena project values security and efficiency over decentralisation.

Kadena emphasizes a security-focused approach while ensuring operational functionality. By centralizing critical functions, the project enhances reliability and rapid decision-making while maintaining accountability, ensuring the system remains robust and efficient.



Centralised

Decentralised

Conclusion

After Hashlock's analysis, the Kadena project seems to have a sound and well-tested code base, now that our vulnerability findings have been resolved and acknowledged. Overall, most of the code is correctly ordered and follows industry best practices. The code is well commented on as well. To the best of our ability, Hashlock is not able to identify any further vulnerabilities.

Our Methodology

Hashlock strives to maintain a transparent working process and to make our audits a collaborative effort. The objective of our security audits is to improve the quality of systems and upcoming projects we review and to aim for sufficient remediation to help protect users and project leaders. Below is the methodology we use in our security audit process.

Manual Code Review:

In manually analysing all of the code, we seek to find any potential issues with code logic, error handling, protocol and header parsing, cryptographic errors, and random number generators. We also watch for areas where more defensive programming could reduce the risk of future mistakes and speed up future audits. Although our primary focus is on the in-scope code, we examine dependency code and behaviour when it is relevant to a particular line of investigation.

Vulnerability Analysis:

Our methodologies include manual code analysis, user interface interaction, and white box penetration testing. We consider the project's website, specifications, and whitepaper (if available) to attain a high-level understanding of what functionality the smart contract under review contains. We then communicate with the developers and founders to gain insight into their vision for the project. We install and deploy the relevant software, exploring the user interactions and roles. While we do this, we brainstorm threat models and attack surfaces. We read design documentation, review other audit results, search for similar projects, examine source code dependencies, skim open issue tickets, and generally investigate details other than the implementation.

Documenting Results:

We undergo a robust, transparent process for analysing potential security vulnerabilities and seeing them through to successful remediation. When a potential issue is discovered, we immediately create an issue entry for it in this document, even though we have not yet verified the feasibility and impact of the issue. This process is vast because we document our suspicions early even if they are later shown to not represent exploitable vulnerabilities. We generally follow a process of first documenting the suspicion with unresolved questions, and then confirming the issue through code analysis, live experimentation, or automated tests. Code analysis is the most tentative, and we strive to provide test code, log captures, or screenshots demonstrating our confirmation. After this, we analyse the feasibility of an attack in a live system.

Suggested Solutions:

We search for immediate mitigations that live deployments can take and finally, we suggest the requirements for remediation engineering for future releases. The mitigation and remediation recommendations should be scrutinised by the developers and deployment engineers, and successful mitigation and remediation is an ongoing collaborative process after we deliver our report, and before the contract details are made public.

Disclaimers

Hashlock's Disclaimer

Hashlock's team has analysed these smart contracts in accordance with the best industry practices at the date of this report, in relation to: cybersecurity vulnerabilities and issues in the smart contract source code, the details of which are disclosed in this report, (Source Code); the Source Code compilation, deployment, and functionality (performing the intended functions).

Due to the fact that the total number of test cases is unlimited, the audit makes no statements or warranties on the security of the code. It also cannot be considered as a sufficient assessment regarding the utility and safety of the code, bug-free status, or any other statements of the contract. While we have done our best in conducting the analysis and producing this report, it is important to note that you should not rely on this report only. We also suggest conducting a bug bounty program to confirm the high level of security of this smart contract.

Hashlock is not responsible for the safety of any funds and is not in any way liable for the security of the project.

Technical Disclaimer

Smart contracts are deployed and executed on a blockchain platform. The platform, its programming language, and other software related to the smart contract can have their own vulnerabilities that can lead to attacks. Thus, the audit can't guarantee the explicit security of the audited smart contracts.

About Hashlock

Hashlock is an Australian-based company aiming to help facilitate the successful widespread adoption of distributed ledger technology. Our key services all have a focus on security, as well as projects that focus on streamlined adoption in the business sector.

Hashlock is excited to continue to grow its partnerships with developers and other web3-oriented companies to collaborate on secure innovation, helping businesses and decentralised entities alike.

Website: hashlock.com.au

Contact: info@hashlock.com.au



#hashlock.